

TIRS FAC – On-Prem Batch Jobs

A high-level summary of the scheduled on-prem batch jobs for the TIRS FAC application — what each job does, where to find its logs, and how the team can check for stuck work and safely re-trigger it after an outage.

No application code changes are needed for any of the recovery steps below — they are database checks and updates only.

1. The big picture

- All **11 jobs** run from a **single scheduled batch application** on-prem, launched by the **Windows Task Scheduler** (each scheduled task starts the same application with a different job name).
- Every job follows the same simple pattern:
 1. **Pick up pending records** from a database table.
 2. **Call an external service** — the DMS document store, SharePoint, Azure Blob storage, or the AI providers (**TRAIT** and **Cytora**).
 3. **Mark the record complete**, or **retry it** if the service call failed.
- Each record is retried only a **limited number of times**. If a downstream service (e.g. Blob storage or DMS) is **down long enough that the retry limit is used up**, the record **stalls** and stops being processed. A short **database script** resets it so the next scheduled run picks it up again.
- The jobs fall into three business groups:

Group	Jobs	What it delivers
-------	------	------------------

Document upload to DMS	1–2	Generates underwriting PDFs / submission documents and files them into DMS.
TRAIT AI extraction	3–8	Reads incoming submission emails through the TRAIT AI service to auto-extract data.
Cytora AI extraction	9–10	Reads submission documents through the Cytora AI service to auto-extract data.
Housekeeping	11	Routine cleanup of AI records and old logs.

2. The jobs at a glance

#	Job name	Purpose	Data a
1	TirsWebFacDMSUpload	Generates underwriting documents (binder / authorization / endorsement letters) and uploads them to DMS.	Docume queue
2	TirsWebFacSubmissionDmsJob	Files submission documents into DMS.	Docume queue

3	TirsWebFac_AI_ApacProfLiabTraitJob	Runs TRAIT AI extraction for the APAC Professional Liability mailbox.	AI queu
4	TirsWebFac_AI_LacProfLiabTraitJob	Runs TRAIT AI extraction for the LAC Professional Liability mailbox.	AI queu
5	TirsWebFac_AI_MedPLFacTraitJob	Runs TRAIT AI extraction for the Medical Professional Liability mailbox.	AI queu
6	TirsWebFac_AI_PLLondonFacTraitJob	Runs TRAIT AI extraction for the Professional Liability – London mailbox.	AI queu
7	TirsWebFac_AI_ProfLiabTraitJob	Runs TRAIT AI extraction for the Professional Liability mailbox.	AI queu

8	TirsWebFac_AI_FailedTraitJob	Automatically re-drives failed TRAIT records (the safety-net retry job).	AI queue
9	TirsWebFac_AI_CytoraJob	Runs Cytora AI extraction for new submission documents.	AI queue
10	TirsWebFac_AI_CytoraFailedJob	Automatically re-drives failed Cytora records (the safety-net retry job).	AI queue
11	TirsWebFac_AI_UtilityJob	Housekeeping — removes duplicate AI records and purges old request logs.	Maintenance

Jobs 3–7 are **the same job pointed at different mailboxes** (one per Professional Liability desk). Jobs 8 and 10 are **retry jobs** that automatically reprocess failures on their own schedule — most transient outages recover without any manual action.

3. On-prem logging

3.1 Where the logs are

All jobs write to a shared network folder, one per environment:

```
\\tirsfiles\tirsdms\{ENV}\FACDMSUploadJob\BatchLogs\
```

{ENV} is one of **DEV**, **QA**, **Test** (UAT) or **Prod**. The path is set per environment in the application configuration (OnPremLogging setting), so each environment keeps its logs separate.

3.2 How the logging works

The jobs use a **per-item logging** approach so activity can be traced back to a single submission:

- **AI jobs (TRAIT & Cytora)** — each processed item is logged to **its own file**, named after the item's reference id: {syselmid}-logs.log. Because syselmid is also a column on the ai_meta_info table, you can copy the id from a database row and open the matching log file directly. Each file records the steps taken for that item (job initiated, polled, completed/failed) and the full error text if something goes wrong.
- **Document jobs (DMS upload / submission)** — these log to a **single daily file** per job: dmsbatch-log-YYYYMMDD for the DMS upload job and submissionbatch-log-YYYYMMDD for the submission job. (The submission spelling is the actual file name on disk.)
- Every line is time-stamped and shows the record keys being worked on (upload / submission id) so a run can be followed end to end.

3.3 Other log copies

- **Azure Blob storage** — when enabled, a copy of the logs is also written to the applog blob container, so they can be reviewed without server access.
- **Local error file** — a rolling Fac-logs/ex_.log file next to the application keeps the last few days of **errors and warnings only**, as a quick fallback.

Both the shared-folder and blob logging are switched on/off per environment through configuration. **If logs are missing, first confirm logging is enabled for that environment** and that the shared folder is reachable from the server running the job.

4. Health checks & recovery (DB/IT)

Two database tables drive all of the jobs:

Group	Table	“Pending” means	“Done” means
Document jobs (1–2)	dms_upload_docs	action_type = 'I' and no DMS location yet	action_type = 'C'
AI jobs (3–10)	ai_meta_info	status_cd is I (new) or P (in progress)	status_cd = 'C'

The recovery idea is always the same: **reset the retry counter (and status) so the next scheduled run reprocesses the record.**

Always run the **SELECT** first to see what will be affected, then run the **UPDATE**. The FAC database is **Informix** (`FIRST n` limits rows).

4.1 Job 1 — TirsWebFacDMSUpload (underwriting documents)

Handles binder / authorization / endorsement letters **and their notification emails** — everything **except** the submission documents handled by job 2. The job runs in two phases: it first generates the documents and uploads them to DMS (`sub_doc_type` is *not* `EMAIL/SUBMISSION`, `dms_location` still empty), then sends the queued `EMAIL` records. The queries below cover **both** phases so failed emails

are also picked up for retrigger.

Check what is pending:

```
SELECT FIRST 200 upload_key, submis_key, doc_type, sub_doc_type,
       retry, lock_timestamp
FROM dms_upload_docs
WHERE action_type = 'I'
      AND (
        -- documents this job generates & uploads to DMS
        (UPPER(sub_doc_type) NOT IN ('EMAIL','SUBMISSION'
        AND (dms_location IS NULL OR dms_location = ''))
        -- notification emails this job sends
        OR UPPER(sub_doc_type) = 'EMAIL'
      )
ORDER BY upload_key;
```

Reset stalled records (retry limit used up, or left “locked” by an interrupted run — e.g. after a Blob / DMS / SharePoint outage):

```
UPDATE dms_upload_docs
SET retry = 0,
    lock_timestamp = NULL
WHERE action_type = 'I'
      AND (
        -- documents this job generates & uploads to DMS
        (UPPER(sub_doc_type) NOT IN ('EMAIL','SUBMISSION'
        AND (dms_location IS NULL OR dms_location = ''))
        -- notification emails this job sends
        OR UPPER(sub_doc_type) = 'EMAIL'
      )
      AND (retry >= 5 OR lock_timestamp IS NOT NULL);
-- Optional: add "AND submis_key = <number>" to fix a sin
```

4.2 Job 2 — TirsWebFacSubmissionDmsJob (submission documents)

Handles only the **submission** documents.

Check what is pending:

```
SELECT FIRST 200 upload_key, submis_key, doc_no, retry, 1
FROM dms_upload_docs
WHERE action_type = 'I'
      AND UPPER(doc_type) = 'SUBMISSION'
      AND UPPER(sub_doc_type) = 'SUBMISSION'
      AND (dms_location IS NULL OR dms_location = '')
ORDER BY upload_key;
```

Reset stalled records:

```
UPDATE dms_upload_docs
SET retry = 0,
    lock_timestamp = NULL
WHERE action_type = 'I'
      AND UPPER(doc_type) = 'SUBMISSION'
      AND UPPER(sub_doc_type) = 'SUBMISSION'
      AND (dms_location IS NULL OR dms_location = '')
      AND (retry >= 5 OR lock_timestamp IS NOT NULL);
-- Optional: add "AND submis_key = <number>" to fix a sin
```

4.3 TRAIT AI jobs (jobs 3–8)

Check what is pending or failed:

```
SELECT FIRST 200 ai_meta_info_key, submis_key, syselmid,
                status_cd, retry_attempts
FROM ai_meta_info
WHERE source_type_cd = 1                -- 1 = TRAIT
      AND status_cd IN ('I','P','F')
      AND (trc_json_response IS NULL OR trc_json_response = '')
ORDER BY ai_meta_info_key;
```

Reset stalled records (retry limit used up after a TRAIT / Blob /

SharePoint outage) so the main TRAIT job reprocesses them from scratch:

```
UPDATE ai_meta_info
SET status_cd = 'I',
    retry_attempts = 0,
    trc_json_response = '',
    orig_json_response = '',
    json_response_file_url = '',
    payload_meta_info = NULL,
    timer_info = NULL,
    export_initiate_dt = NULL
WHERE source_type_cd = 1                -- 1 = TRAIT
    AND status_cd IN ('F','P')
    AND (trc_json_response IS NULL OR trc_json_response = ''
    AND retry_attempts > 5;
```

To reset only **one Professional Liability desk / mailbox**, first list the TRAIT mailboxes...

```
SELECT ai_provider_info_key, provider_name, axto AS mailbox
FROM ai_provider_info
WHERE provider_name = 'TRAIT'
ORDER BY axto;
```

...then add `AND ai_provider_info_key = <key>` to the reset above.

4.4 Cytora AI jobs (jobs 9–10)

Check what is pending or failed:

```
SELECT FIRST 200 ai_meta_info_key, submis_key, syselmid,
    status_cd, retry_attempts
FROM ai_meta_info
WHERE source_type_cd = 2                -- 2 = Cytora
    AND status_cd IN ('I','P','F')
```

```
AND (trc_json_response IS NULL OR trc_json_response = '  
ORDER BY ai_meta_info_key;
```

Reset stalled records (retry limit used up after a Cytora / Blob / SharePoint outage) so the main Cytora job reprocesses them from scratch:

```
UPDATE ai_meta_info  
SET status_cd = 'I',  
    retry_attempts = 0,  
    trc_json_response = '',  
    orig_json_response = '',  
    json_response_file_url = '',  
    payload_meta_info = NULL,  
    timer_info = NULL,  
    export_initiate_dt = NULL  
WHERE source_type_cd = 2                -- 2 = Cytora  
    AND status_cd IN ('F','P')  
    AND (trc_json_response IS NULL OR trc_json_response = '  
    AND retry_attempts > 5;
```

5. Things to watch when changing the database

Plain-language cautions for anyone editing these tables or planning schema changes:

- **Reset the counter *and* the status together.** Clearing only the status but leaving the retry counter maxed out means the job still skips the record.
- **Clear “locked” document records after a crash.** If a run is interrupted, affected `dms_upload_docs` rows stay flagged as in-use and are skipped until the lock is cleared — there is no automatic timeout.
- **Don’t mark records “complete” to stop them.** Setting the

completed status hides a record permanently; there is no job to undo it.

- **Mailbox mapping matters.** The five TRAIT jobs are separated only by mailbox. Renaming or repointing a mailbox can silently stall an entire desk's queue.
 - **Involve the development team for schema changes.** Adding, renaming, or changing the meaning of columns on `dms_upload_docs`, `ai_meta_info` or `ai_provider_info` can stop jobs from finding their work. Coordinate before applying such changes.
 - These reset scripts are **operational fixes**, not permanent database changes.
-

6. Quick reference

- **Document status** (`dms_upload_docs.action_type`): `I` = pending · `C` = completed.
- **AI status** (`ai_meta_info.status_cd`): `I` = new · `P` = in progress · `C` = completed · `F` = failed.
- **AI provider** (`ai_meta_info.source_type_cd`): `1` = TRAIT · `2` = Cytora.
- **Retry limits:** document jobs and main AI jobs retry up to ~5 times; the dedicated “failed” AI jobs retry up to ~3 times before a record needs a manual reset.